

LLMs are Easily Startled: Fragility Testing with Lookalike Character Noise

Peter deJonge¹ and Nathanael Chambers^{1,2}

¹US Naval Academy, Annapolis MD

²Williams College, Williamstown MA
nc19@williams.edu

Abstract

Language models are dependent on the character sets of the languages seen during training. Some languages use large character sets, like Chinese and Korean, while others are very small, like English. When the input during inference diverges from the trained character set, it can lead to unexpected behavior. While many researchers have looked at using this weakness for adversarial attacks, these studies typically focus on extreme cases that lead to model failure. But we don't have a strong understanding of how brittle models are to more casual and routine character substitutions. In fact, many 'known' characters have Unicode lookalikes that users enjoy using without malicious purposes. This paper studies the *fragility* of models when faced with such character noise. When input contains innocuous character substitutions, do models robustly handle the distortion? How does the tokenizer and overall model size affect performance? This paper presents analyses of tokenizers and a series of fragility tests to answer these questions. We find that tokenization is not the only stumbling block, and neither is a larger model size the assured solution. We present a formal benchmark, publicly release a Fragility Scorer¹, and describe our experimental results on a variety of open-source models. Implications and suggestions for improving fragility are then presented.

1 Introduction

Language models are trained on sequences of tokenized text, and a wealth of research has shown that input which contains unexpected formatting that diverges from training formats can lead to strange behavior. One particular kind of formatting is lookalike character substitution ($\rho \rightarrow p$). These are used in a lighthearted manner on social media, or for more intentional information hiding

purposes that humans easily read, but which fool automated extractors. This paper studies what we call the "fragility" of models when facing general character distortion and manipulation. In particular, we look at how model size and its tokenizers affect robustness.

A number of research efforts have focused on manipulating model behavior by altering its input. These include jailbreaking (Yi et al., 2024), adversarial prompting (Chao et al., 2025), synonym swapping (Li et al., 2024), gradient-based attacks (Ren et al., 2025), and many others. Most of these focus on an intentional goal of altering system behavior, thus proposing methods to discover inputs to produce maximum impact. This paper, however, broadens the aperture of study to the *general* performance of models in the face of character manipulation. While a model might fail on a specific attack, it could still be robust to 99% of manipulations that occur in typical digital writing.

The goals of this work are partially motivated by recent findings that large language models (LLMs) struggle on simple character and word-level challenges. Efrat et al. (2023) created a series of tests for word manipulation (e.g., 'what word occurs after "the" in the following sentence') that illuminated the struggle between model tokenization and lexical manipulation that humans perform with ease. Despite the many strengths of LLMs on complex tasks, they struggle on 'easy' ones. We thus look deeper into character-level changes and how they might affect downstream complex tasks.

Several societal challenges also motivate the importance of understanding model fragility. We take inspiration from human trafficking prevention where Unicode character swapping is observed in illegal online advertisements. Phone numbers are sometimes masked with digit lookalikes (e.g., $\emptyset$ not 0). Again, the malicious actor isn't trying to break a model per se, but instead just hopes to hide the digits. Can LLMs still extract informa-

¹<https://www.github.com/nchambers/fragility>

tion from text that diverges from its trained character set? How fragile are today’s models? Does model size correlate with fragility? We investigate these questions through a series of fragility tests.

The main contributions of this work include: (1) A carefully chosen set of character swapping tests to evaluate model fragility, (2) Fragility scores from 7 families of open-source models across 24 size variants, (3) Results from a variety of prompting approaches, from few-shot to the inclusion of unicode mappings (e.g., telling the model that α is a), (4) The first analysis of how Unicode substitutes are represented by tokenizers in comparison to their ASCII lookalikes, (5) A task for phone number extraction motivated by the domain of human trafficking, and (6) Release of the Fragility Scorer software to test future models.

In summary, this paper provides analyses on how Unicode lookalikes are represented in Transformer-based models, how noise affects performance, how model size affects fragility, and how today’s models compare to each other.

2 Related Work

Research into adversarial attacks on LLMs is related to this paper, but tends to focus more on inputs that achieve specific behavior. Jailbreaking manipulates the prompt to circumvent model instructions. A variety of helpful survey papers are available (Knowlton et al., 2026; Yi et al., 2024; Tian et al., 2025; Das et al., 2025). Gradient-based attacks access model parameters to discover input tokens that most affect the final distribution (Li et al., 2018, 2020; Zou et al., 2023; Ren et al., 2025), crafting attacks that alter model output. Many attacks add instructions to the prompt (Liu et al., 2024; Chao et al., 2025), some using multilingual substitutions (Deng et al., 2024). One approach that is a bit closer to this paper is synonym swapping: searching for input tokens to replace for maximum effect (Li et al., 2024; Wang et al., 2024; Zheng et al., 2025b). A first look at homoglyph attacks on encoders came from Boucher et al. (2022). We differ from the bulk of attack-based research in that we are not trying to break systems nor are we trying to find the *best* attack, but rather we seek to measure overall fragility of models when facing general character manipulation, some of which is innocuous and unintentional.

Most relevant to this paper is prior work on character substitutions. There is a nice thread

of recent work on how traditional *classifiers* (not LLM generation) are affected by Unicode or other character substitutions. Edman et al. (2024) proposed a series of character *tasks* for LLMs to perform, such as swapping letters and reversing words. CharBench is a similar set of tasks for character analysis (Uzan and Pinter, 2026). Yuan et al. (2024) study character cipher substitutions. A number of people have looked at LLM awareness of substrings (Ciaccio et al., 2025; Uzan and Pinter, 2026). Some have studied tokenization and how breaking up tokens in English (Zheng et al., 2025a) and Chinese (Haslett, 2025) affects LLM performance. Zhang et al. (2021) was one of the first studies of how BERT-based classifiers perform when training/testing with character substitutions. They found that performance was poor, but could be improved with training data augmentation. This is obviously not an option for today’s largest models without direct access to computational resources beyond most researchers.

Finally, work on visual models proposes new architectures that use CNNs to build the input characters’ visual embeddings (Liu et al., 2017; He et al., 2018; Summers et al., 2023), thus learning to identify Unicode and ASCII similarity more like a human’s eye (Eger et al., 2019; Salesky et al., 2021; Rust et al., 2023). The idea is not to use character (or token) embeddings as the input, but visual images that are transformed by CNNs. While an interesting research area, this paper does not focus on CNN architectures or building custom LLM frameworks, but instead on the standard Transformer of today’s most used models.

3 Fragility Tests

This section describes our series of fragility tests to stress-test language models for robustness in character noise. We separate these into three types of tests: letters, digits, and broader language understanding. The digit tests are grounded in the extrinsic task of phone number identification. The letter tests are grounded in word identification, and understanding uses a common Q&A benchmark that tests language understanding ability.

3.1 Lookalike Dataset

For all experiments in this paper, we use the open-source homoglyph dataset². Specifically, we use its raw character clusters that represent lookalike

²www.github.com/codebox/homoglyph

MMLU	MMLU	Answer this question: ... (A) possible answer ...
	MMLU lookalike	Answer this question: ... (A) possible answer ...
Characters	What is char	What is the ASCII letter most similar to this? <i>i</i>
	Char blanks	What is this word: p_??_s
	Char lookalike	What is this word: pizzas
	Word Filler	What is this word: p..izz**as
	Word Filler+look	What is this word: p..izz**as
	What is sent	What is this sentence: The 6orse jumped over the fence.
Digits	What is char	What is the ASCII digit most similar to this? 3
	Middle digit	What is this phone number: 4108763843
	Multiple digits	What is this phone number: 4108763843
	Digit Spelling	What is this phone number: 41zero876three843
	Number Filler	What is this phone number: 41..08-76,,3843
	Num Filler+look	What is this phone number: 41..08-76,,3843

Figure 1: The subtasks in the Fragility Score.

Unicode characters based on the ASCII digits 0-9 and letters a-z and A-Z. Its creators constructed this dataset by manually identifying all of the lookalikes they could find. We did not augment or alter the dataset, protecting from any bias from this paper’s authors. An example of the q character lookalikes is shown here:

qqqq q qqqq qqqq qqqqqq

3.2 Baseline Character Identification

We start with a baseline fragility test that asks in the most straightforward manner, *what is this character?* We query the LLM to identify which ASCII character is most similar (and visual) to the given Unicode character. We include two separate baseline tests that instead specifically ask for a digit or a letter, as appropriate. For instance:

What ASCII digit is most similar
to this unicode character: 3

The correct answer is obviously 3. We provide three examples as a few-shot prompt so smaller models (and models without instruction tuning) have their best chance to demonstrate its learned character set knowledge. See Appendix A.1 for more prompts.

3.3 Letter Understanding

Identify the Word with Blanks: This baseline test evaluates a model’s ability to "fill in the blanks" when given a word with missing letters.

p_??_s → pizzas

This basic test sets the stage for the next evaluation that inserts Unicode lookalikes into the blanks.

A model’s performance on this blank test measures a model’s ability to identify a partial word with no extra information.

We selected 300 English words of length 6 at random. Two of the characters are randomly removed and replaced with underscore characters. This word length was chosen to balance rarity (longer words are easier to guess) with having a good mix of visible and hidden characters.

Identify the Word with Lookalikes: This test uses the same words as above, but instead of underscore characters, Unicode lookalikes are substituted into the underscore locations:

pizzas — pizzas

If a model has a learned representation of lookalike character usage, it should perform better on this test than the prior underscore test.

Identify the Word with Noise: This test uses the same 300 words in the above tests, but with all the original ASCII. The test then inserts punctuation between randomly selected characters in the words, testing a model’s ability to understand a word in the presence of confounding noise.

We created two versions of this test. One plain test with just ASCII and noise, and the other uses the Lookalikes test above with added noise. The former is called *Word Noise*, and the latter *Noise+Look*.

Identify the Sentence with Lookalikes: While the word tests above give the most straightforward insight into model understanding of letters in context, the sentence-level test gives the model *more* context by allowing the surrounding words to assist the LLM. This test is given a sentence with lookalike character swaps, and the model must repeat the sentence in ASCII form. To provide the optimal environment for success, we also provide three few-shot examples in the prompt. One example sentence is here:

The 6orse jumped over the fence.

A correct answer is a generation of the sentence (case-insensitive) with only ASCII characters. The scorer does not penalize for extra whitespace.

3.4 Digit Understanding

The fragility tests for digit recognition are grounded in a real-world task, rather than generic digit prompts. We chose the task of phone number extraction, motivated by its applicability to

downstream applications. One noteworthy application area is human trafficking prevention where phone numbers are the strongest indicator for linking online advertisements (Hundman et al., 2018; Rodríguez Díaz, 2021; Vajiac et al., 2023; Saxena et al., 2023). Phone numbers also serve well as an evaluation because LLMs are thus primed to generate 10 digits.

Number Recognition: Middle Digit This first test provides a phone number where the middle digit has been replaced with a Unicode lookalike character. We prompt the LLM to generate the original phone number from this perturbed input in its entirety. This allows us to evaluate not just that the middle digit is recovered, but also that the model does not confuse the generation of the surrounding ASCII digits either.

Number Recognition: Multiple Digits This test randomly substitutes 33% of the digits with a lookalike character. We run several versions of this test with increasing percentages.

Number Recognition: Noise This test evaluates the fragility of models when the input is not ‘properly’ formatted, but instead contains extra whitespace and punctuation between characters, such as in ‘4=1..0-93,485 31’.

Number Recognition: Semantic Substitution This test replaces digits with spelled out versions (3 → three), including a random choice of lowercase and uppercase characters (3 → tHReE). Different from the prior tests, only ASCII characters are used, but this test measures the fragility of models when forced to sequentially understand character sequences as digits (four5one vs 451).

Combinations Finally, the fragility tests can create combinations of the above, such as a test that includes both Multiple Digits and Noise. We present results on this at different percent likelihoods in the data.

3.5 Language Understanding

While the above tests evaluate word-level handling, they don’t evaluate broader understanding and they don’t directly measure how semantic understanding is affected by character noise.

Our goal is to measure model performance on common Q&A benchmarks before and after lookalike substitutions are made. However we do not perturb the benchmark text in its entirety, but rather keep the questions untouched, allowing the

The surface of Planet Earth loses energy to outer space due mostly to

- A. conduction.
- B. convection.
- C. radiation.
- D. radioactivity.

Figure 2: Example MMLU question whose answers contain Unicode noise. The question is unchanged.

model to fully understand the question. This fragility evaluation only alters the multiple-choice *answers*, keeping the question clear. We test a model’s ability to extract information from it. See Figure 2 for an example.

We evaluate fragility of Q&A using the 57 subject MMLU benchmark (Hendrycks et al., 2021). For each LLM, we measure its current accuracy without text changes, and then again but with 20% of the characters in the answers swapped with lookalike substitutes. The score is computed as $(1 - acc20/acc)$. If a model struggles to generate the answer in an extractable format, it is marked incorrect, but the ratio only penalizes on the noise’s effect. See Appendix A.5 for more details.

3.6 Chain of Dictionary

Recent work in machine translation experimented with the idea of prompting models with translation equivalents (Lu et al., 2024). For instance, insert ‘casa means house’ into the prompt. We adopt this idea, called *chain of dictionary* (CoD), for lookalikes with the homoglyph replacements in the prompt: ‘I is 1’ if the character ‘I’ appears in the phone number. The digit tests in Figure 1 are duplicated, but with CoD hints, and we separately record their CoD accuracy scores.

4 Tokenizer Analysis

Before the input to an LLM is transformed through its neural layers, the immediate effect of noisy input is observed in how the model’s tokenizer splits the text. When unknown characters are in the input, a sentence that might normally be tokenized into a few tokens can expand into 20+ tokens even if just a few Unicode characters are present. An example with Gemma’s tokenizer is shown in Figure 3.

Our initial experiments are a set of exploratory tokenization and token embedding measurements to develop an assessment of each tokenizer that

The Little Thoughts: The Little Thoughts
 The Little Thoughts: The Little Th????hts

Figure 3: Gemma’s tokenizer: altering 3 characters results in 12 tokens vice 3. Question marks show where the 4 bytes in unicode ‘o’ are tokenized separately.

can then be compared to actual fragility performance. A tokenizer that requires more tokens than another is not necessarily worse off; its network layers could recover meaning from a fragmented input. This analysis measures the correlation between tokenization and performance.

We analyze tokenizers from Llama (also Vicuna), Qwen (also Deepseek), Mistral, Gemma, & OLMo. Our focus is along three dimensions:

- **Token recognition:** what percent of Unicode lookalikes appear in the tokenizer?
- **Embedding similarity:** are Unicode embeddings (as tokens) similar to their nearest-neighbor ASCII characters?
- **Fertility:** how does tokenization increase with character substitutions?

For **token recognition**, we counted how many Unicode lookalikes appear in the tokenizer as standalone tokens or with a whitespace prefix. Figure 4 shows the percent found in the tokenizer (blue bars). There is a clear and noticeable difference in tokenization, with Gemma and Qwen (Deepseek uses Qwen) representing 50% of them while the others are less than 10%. We then measured **embedding similarity** of these tokens to see if their embeddings capture ASCII similarity (i.e., is the Unicode character similar to its visual ASCII equivalent?). We find the Top 3 most similar other tokens using cosine similarity, and see if the ASCII lookalike is among them. The orange bar shows that Gemma and Qwen have learned this similarity, but interestingly, Deepseek’s further training of Qwen drastically reduced it.

Finally, we measured the **fertility** of tokenizers that experience tokenization explosion when an input contains lookalikes (Figure 4 shows how one example affects Gemma). This experiment took 2000 examples from the MMLU benchmark, and tokenized them with and without 20% of the characters replaced with Unicode lookalikes. We compute the average number of tokens with and without. Figure 5 shows the averages from each tokenizer. The dark shaded interior shows that they

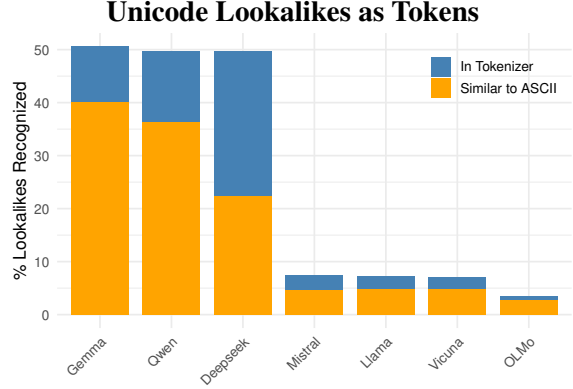


Figure 4: Percent of unicode lookalikes that appear in tokenizers as a standalone token (blue+orange). The orange bar indicates how many known tokens have an embedding similar to its ASCII lookalike. Qwen and Deepseek(Qwen-based) use the same tokenizer, but their embeddings are different.

Model	Sizes Tested
Gemma 3	1B, 4B, 12B, 27B
Llama 3.2	1B, 3B, 70B
OLMo 2	1B, 7B, 13B, 32B
Vicuna 1.5	7B, 13B
Deepseek R1-Qwen	1.5B, 7B, 14B, 32B
Qwen 3	.6B, 1.7B, 4B, 8B, 14B
Mistral-Small	23B
Mixtral-8x7B	7B/56B
GPT-4.1, 5.4, 5.5, 4.1mini, 4omini, 5.4mini, 5mini	unknown

Table 1: Models scored by the Fragility Benchmark.

are similar on regular ASCII text, but lookalike noise causes varying amounts of tokenization explosion. Vicuna and Mistral experience the most drastic expansion requiring 250% more tokens.

We discuss this tokenizer analysis in context of the fragility results after the next section.

5 Experimental Setup

5.1 Models

Table 1 lists the open-source models and sizes used in our experiments. We include both small and medium-sized models to assess how size may or may not affect a Transformer architecture’s ability to handle character substitutions.

We also include several models from the OpenAI suite to provide a general comparison against mini and large closed-source models. Other closed LLMs exist, but this work is not an engineering experiment to stress-test black box models. Regrettably, the labs and corporations behind these models have not recently released their latest model ar-

Number of Tokens by Tokenizer

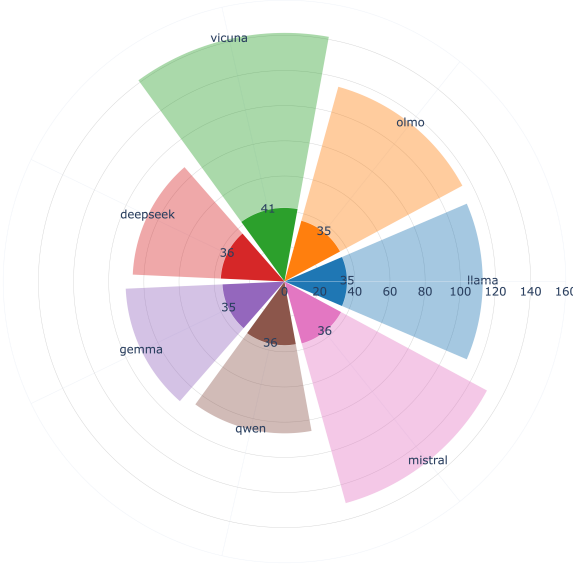


Figure 5: Visualization of how many tokens are produced on normal text (dark shaded) and when 20% is replaced with Unicode lookalikes (lighter). Vicuna and Mistral produce over 250% more tokens.

chitecture changes, training data, nor training procedures. Since few meaningful conclusions from their performance can be drawn without an open release of their architecture, we do not expend computational resources beyond these models. As will be seen, flagship closed-source models also struggle with fragility.

5.2 Fragility Tests and Prompting

We prompt each model (in Table 1) for each test within the Fragility Score (in Section 3) and report individual results for the main tests, as well as one aggregate fragility score.

The prompt templates for each test were chosen based on author experimentation using general instruction language. We altered the prompts with more instructions until generation was reliable in format across the models. Most importantly, we created these prompts using several different models during initial development. The final prompts used for each test are provided in appendix A.1.

The Fragility Score (FS) is a linear combination of accuracies over the three subsets of test types:

$$FS = \lambda_L Letter + \lambda_D Digit + \lambda_U NLU \quad (1)$$

where the lambda weights sum to one and the sub-scores are themselves linear combinations of individual test accuracies (each accuracy ranges [0, 100]). We report with $\lambda_L = 0.3$, $\lambda_D = 0.3$,

$\lambda_U = 0.4$, but this formulation allows for adjusting the weights to any task balance.

6 Results

All model Fragility Scores can be seen in Figures 6 and 7. At a high-level, there is a trend of an improved fragility score as model size increases, but it is not consistent across all model types. There is a noticeable cluster of models that have fewer parameters yet display much more robustness in the fragility score than the others (circled in the scatterplot). Gemma and Qwen stand out as more robust starting at the ~4B model size. Qwen has a straight line of performers no matter the model size, as does Gemma. You can also see 14B Deepseek in the circle, and its larger 32B model doesn’t improve further. Model size is clearly not the sole factor determining fragility. See Appendix A.3 for establishing confidence in these scores.

Figure 6 provides more insight into the different fragility components. We see the Qwen family as the strongest set of models on overall Fragility. Its small 4B model shines amidst the other top performers with larger 14-32b sizes. Gemma is not far behind. But not all models are consistent across the subparts. Several are fragile on digits despite being more robust on letters. The OLMo 32b model is one of the bigger models, and arguably the best on NLU/text, but it struggles as an outlier on digit handling.

Are looklike characters always disruptive?

We compared the two ‘what is this word’ tests (underscores versus Unicode) to better understand how the model makes its predictions. Interestingly, the Unicode results are substantially higher than the underscore results across all of the models, often with 2x the accuracy. This illustrates how lookalikes are not purely noise, and are more informative than blanks. Appendix A.4 provides more details. We also analyze the effects of punctuation noise in appendix A.6. The median change in phone number prediction from 20% to 80% added noise is a 47% accuracy drop.

How is NLU affected? The MMLU benchmark subscore is with 20% character replacement, but we also tested 40% and 60% in Figure 8. Mistral and Vicuna see the largest drops, effectively breaking their NLU capability. The best tokenizing models do better, but it’s not consistent. Qwen drops almost 50% on NLU, despite it outperforming on other tasks. It’s clear that tokenization is

gemma27b	76	81	73	76
mistral_small_in	79	84	66	75
gemma12b	74	90	65	75
deepseek32b	75	79	63	72
qwen14b	82	68	65	71
qwen4b	69	80	58	68
olmo32b	86	87	37	67
qwen8b	73	67	60	66
deepseek14b	74	74	52	65
gemma4b	79	73	49	65
mistral_small	78	49	66	64
qwen1.7b	50	65	38	50
mixtral7b	51	21	55	44
olmo13b	54	68	13	42
llama70b	39	74	19	41
deepseek7b	46	32	40	40
qwen0.6b	34	61	19	36
gemma1b	41	49	23	36
vicuna13b	46	24	34	35
olmo7b	43	58	9	34
deepseek1.5b	27	38	21	28
llama3b	47	28	14	28
vicuna7b	34	7	21	20
olmo1b	12	44	3	18
llama1b	28	13	6	15
phi2	11	26	1	11
	Letters	NLU	Digits	Overall Score

Figure 6: Model scores on individual components of the Fragility Score. [0-1] shown as [0-100] for readability.

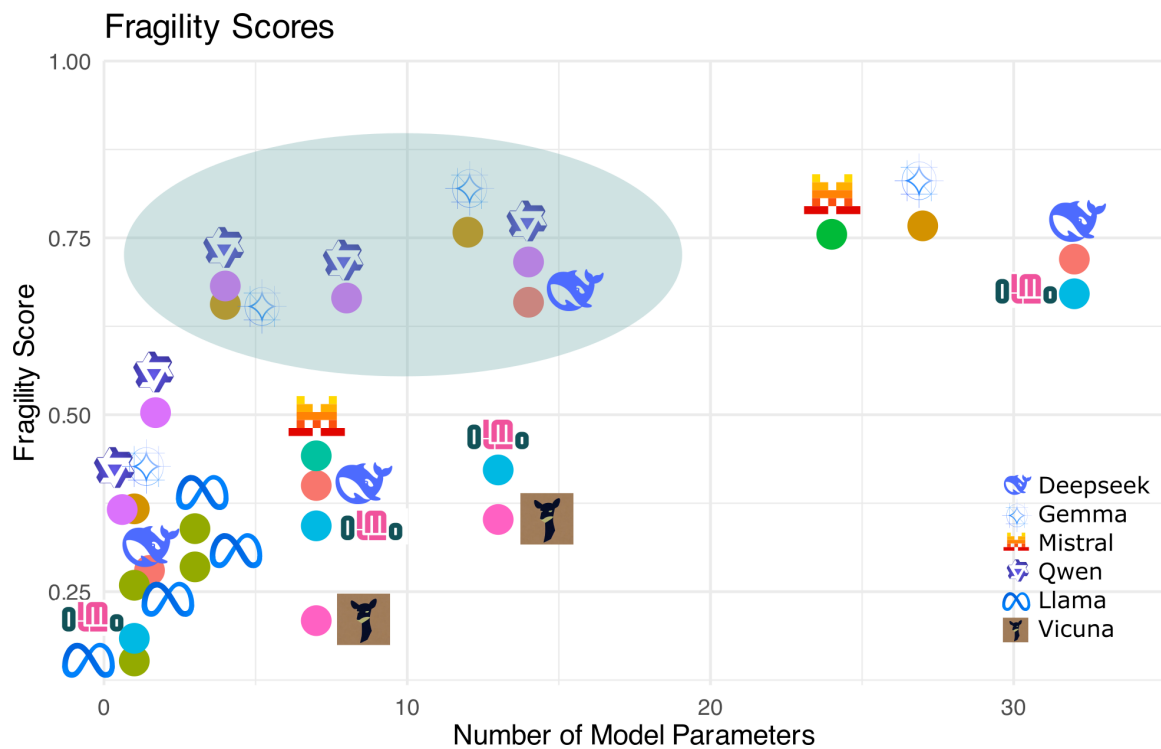


Figure 7: Fragility score for each model, comparing model size (in billions) to its score. The circle indicates outliers that outperform despite having fewer parameters. Qwen and Gemma are less fragile even in smaller sizes.

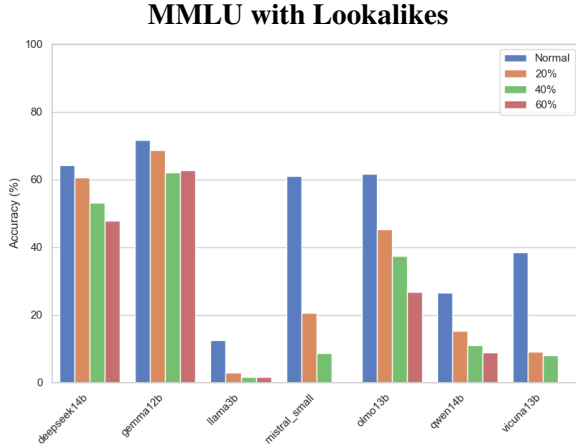


Figure 8: MMLU accuracy at different amounts of characters in the answers replaced with Unicode lookalikes.

not the only piece relevant to handling noise.

Frontier Model Comparison. With both "small" and large GPT models, these results are not truly comparable because we don't have knowledge of additional "reasoning" that the models generate behind the scenes. Some of them are also known to have post-generation classifiers and re-generation, so their algorithms cannot be compared. However, it can help identify possible ceiling scores on Fragility. Figure 9 shows GPT fragility. The largest models are less fragile in the 80's with the "small" GPT models in the 70's, *similar to the best open-source small models*. We include Gemma-12b for comparison. This is significant because the small GPT models are assumed to be larger than these open-source sizes, yet we see similar fragility in them.

Tokenization vs Model Size. To help determine if tokenization (Figure 4) or model (log) size can predict fragility score, we ran a regression analysis. Individually, each shows weak correlation with $R^2 = 0.26$ and 0.42 respectively. However, both predictors in a multiple regression model reveal a strong relationship with fragility: $R^2 = 0.73$, $F(2, 24) = 33.3$, $p < .001$.

7 Discussion

What is the core driver of model fragility? Our results reveal that both tokenization and model size are key, but its both together that are most predictive of fragility performance.

Our tokenization studies, illustrated in Figure 5, show that input recognition is important where Gemma's and Qwen's tokenizers are clear win-

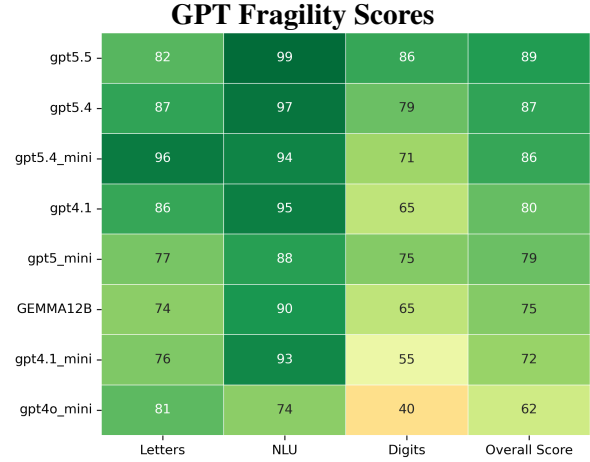


Figure 9: GPT Fragility with Gemma for comparison. Mini models are unknown sizes and are likely MoE.

ners, performing better on fragility despite fewer parameters. Their tokenizers simply contain more of the lookalikes than the other models. However it's clear that training differences also matter. We see this in Deepseek's Qwen-based tokenizer: while they share tokenization, Deepseek's further training has shifted their embeddings away from ASCII lookalikes, worsening fragility. Future work should investigate why/how.

We also found through statistical tests that the added parameterization within larger models, such as Mistral and OLMo-32b, can allow for reconstruction of a scattered input into a meaningful internal representation. The 32b variant of OLMo, which is also the tokenizer with the *fewest* recognized Unicode characters, has a similar fragility score to the smaller Qwen-4b. Qwen's tokenizer clearly gives it an initial boost to model fragility, but larger parameterization is also capable of catching up later. Our R^2 results confirm this.

These results show that tokenizer recognition of lookalikes is important, and model size can compensate some for missing representation. However, the statistical analysis shows that the two together are most crucial. We recommend intentionally adding Unicode lookalikes to tokenizers as a first step, but urge the need for future research in how to fine-tune for this noise.

We hope this paper leads to further studies in broad fragility, not just for adversarial attacks. Since our Fragility Score is a linear combination of a diversity of tests, it can be customized to different task needs. The Fragility Scorer for LLMs is available with tokenizer analysis code at github.com/nchambers/fragility.

Limitations

Unicode character sets are very large and continually expanding with new versions. Our set of hand-identified Unicode lookalike characters is likely not complete, and so these experiments may have missed some characters that would alter the fragility score. However, we intentionally used an external dataset for our experimental set of lookalikes to avoid author bias in what should be included.

This comparison of model fragility does not include all of the closed-source models that are available at the time of writing. Closed-source models are trained on unknown datasets, most privately owned, and architectures are generally not communicated to the scientific community. It is possible that these models contain fundamental advances to assist in character understanding, but determining how or what contributes is not possible by the research community. We include in our results the evaluation of several GPT models, large and small, as data points for the curious reader.

The weighted factors that make up the Fragility Score are chosen to balance character, digit, and overall NLU performance, but these weights are not an empirically motivated decision. Other weights may be more appropriate for any given task, and we believe this flexibility within the Fragility Score is a positive feature.

Ethical Considerations

This paper is a study of how and why LLMs are affected by noisy input at the character level. While much research has focused on the *how*, such as through intentional adversarial attacks, this paper focuses more on the *why* through analysis of model size and tokenization. While identifying these weaknesses carries a risk of dual-use exploitation, a better understanding of these failure modes is essential for building defensive safeguards. We hope that this work is a net benefit to research in assistive systems and more resilient models.

Our study of phone numbers did not intentionally include real numbers, but instead were artificially generated with a random process constrained by the rules of US-based numbers. No human data was used. However, given the saturation of the phone number space, it is possible, even likely, that real numbers were randomly generated and included in the dataset.

Acknowledgments

This work was supported by funding from the Irregular Warfare Technical Support Directorate (IWTSD). The views expressed in this paper are those of the authors and do not reflect the official policy or position of the IWTSD, U.S. Naval Academy, Department of the Navy, the Department of Defense, or the U.S. Government.

References

- Nicholas Boucher, Ilia Shumailov, Ross Anderson, and Nicolas Papernot. 2022. [Bad characters: Imperceptible nlp attacks](#). In *Proceedings of the 2022 IEEE Symposium on Security and Privacy (SP)*, pages 1987–2004.
- Patrick Chao, Alexander Robey, Edgar Dobriban, Hamed Hassani, George J. Pappas, and Eric Wong. 2025. [Jailbreaking black box large language models in twenty queries](#). In *2025 IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*, pages 23–42.
- Cristiano Ciaccio, Marta Sartor, Alessio Miaschi, and Felice Dell’Orletta. 2025. [Beyond the spelling miracle: Investigating substring awareness in character-blind language models](#). In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 11361–11372, Vienna, Austria. Association for Computational Linguistics.
- Badhan Chandra Das, M Hadi Amini, and Yanzhao Wu. 2025. Security and privacy challenges of large language models: A survey. *ACM Computing Surveys*, 57(6):1–39.
- Yue Deng, Wenxuan Zhang, Sinno Jialin Pan, and Lidong Bing. 2024. Multilingual jailbreak challenges in large language models. In *International Conference on Learning Representations*, volume 2024, pages 24634–24651.
- Lukas Edman, Helmut Schmid, and Alexander Fraser. 2024. [CUTE: Measuring LLMs’ understanding of their tokens](#). In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 3017–3026, Miami, Florida, USA. Association for Computational Linguistics.
- Avia Efrat, Or Honovich, and Omer Levy. 2023. [LMentry: A language model benchmark of elementary language tasks](#). In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 10476–10501, Toronto, Canada. Association for Computational Linguistics.
- Steffen Eger, Gözde Gül Şahin, Andreas Rücklé, Ji-Ung Lee, Claudia Schulz, Mohsen Mesgar, Krishnakant Swarnkar, Edwin Simpson, and Iryna Gurevych. 2019. [Text processing like humans do: Visually attacking and shielding NLP systems](#). In

- Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 1634–1647, Minneapolis, Minnesota. Association for Computational Linguistics.
- David A. Haslett. 2025. [Tokenization changes meaning in large language models: Evidence from chinese](#). *Computational Linguistics*, 51(3):785–814.
- Linchao He, Dejun Zhang, Long Tian, Han Fei, Mengting Luo, Yilin Chen, and Yiqi Wu. 2018. [Visual-Based Character Embedding via Principal Component Analysis: 4th International Conference of Pioneering Computer Scientists, Engineers and Educators, ICPCSEE 2018, Zhengzhou, China, September 21-23, 2018, Proceedings, Part I](#), pages 212–224.
- Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2021. [Measuring massive multitask language understanding](#). In *The Ninth International Conference on Learning Representations*, Virtual.
- Kyle Hundman, Thamme Gowda, Mayank Kejriwal, and Benedikt Boecking. 2018. Always lurking: Understanding and mitigating bias in online human trafficking detection. In *Proceedings of the 2018 AAAI/ACM Conference on AI, Ethics, and Society*, pages 137–143.
- Brynn Knowlton, Jovani Campa, David Solis Gallo, Khalil Dajani, and Nabeel Alzahrani. 2026. [Prompt-based jailbreaking of leading LLM chatbots: A survey of attacks and defenses](#). *IEEE Transactions on Artificial Intelligence*, pages 1–15.
- Jinfeng Li, Shouling Ji, Tianyu Du, Bo Li, and Ting Wang. 2018. Textbugger: Generating adversarial text against real-world applications. *arXiv preprint arXiv:1812.05271*.
- Linyang Li, Ruotian Ma, Qipeng Guo, Xiangyang Xue, and Xipeng Qiu. 2020. Bert-attack: Adversarial attack against bert using bert. *arXiv preprint arXiv:2004.09984*.
- Xirui Li, Ruochen Wang, Minhao Cheng, Tianyi Zhou, and Cho-Jui Hsieh. 2024. [Drattack: Prompt decomposition and reconstruction makes powerful LLM jailbreakers](#). *arXiv preprint arXiv:2402.16914*.
- Frederick Liu, Han Lu, Chieh Lo, and Graham Neubig. 2017. [Learning character-level compositionality with visual features](#). In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 2059–2068, Vancouver, Canada. Association for Computational Linguistics.
- Xiaogeng Liu, Nan Xu, Muhao Chen, and Chaowei Xiao. 2024. [Autodan: Generating stealthy jailbreak prompts on aligned large language models](#). In *International Conference on Learning Representations*, volume 2024, pages 56174–56194.
- Hongyuan Lu, Haoran Yang, Haoyang Huang, Dongdong Zhang, Wai Lam, and Furu Wei. 2024. [Chain-of-dictionary prompting elicits translation in large language models](#). In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 958–976, Miami, Florida, USA. Association for Computational Linguistics.
- Qibing Ren, Hao Li, Dongrui Liu, Zhanxu Xie, Xiaoya Lu, Yu Qiao, Lei Sha, Junchi Yan, Lizhuang Ma, and Jing Shao. 2025. [LLMs know their vulnerabilities: Uncover safety gaps through natural distribution shifts](#). In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 24763–24785, Vienna, Austria. Association for Computational Linguistics.
- Paula Rodríguez Díaz. 2021. Identifying sources of human trafficking within online escort advertisements written in spanish. Master’s thesis, Universidad de los Andes.
- Phillip Rust, Jonas F. Lotz, Emanuele Bugliarello, Elizabeth Salesky, Miryam de Lhoneux, and Desmond Elliott. 2023. [Language modelling with pixels](#). In *The Eleventh International Conference on Learning Representations*, Kigali, Rwanda.
- Elizabeth Salesky, David Etter, and Matt Post. 2021. Robust open-vocabulary translation from visual text representations. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*.
- Vageesh Saxena, Benjamin Ashpole, Gijs van Dijck, and Gerasimos Spanakis. 2023. [IDTraffickers: An authorship attribution dataset to link and connect potential human-trafficking operations on text escort advertisements](#). In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 8444–8464, Singapore. Association for Computational Linguistics.
- Lucía Summers, Alyssa Shallenberger, John Cruz, and Lawrence Fulton. 2023. [A multi-input machine learning approach to classifying sex trafficking from online escort advertisements](#). *Machine Learning and Knowledge Extraction*, 5:460–472.
- Yudian Tian, Dongcheng Li, and Siqing Xue. 2025. A survey of jailbreaking attacks on large language and multimodal models. In *Proceedings of the 2025 8th Artificial Intelligence and Cloud Computing Conference*, pages 142–150.
- Omri Uzan and Yuval Pinter. 2026. [CharBench: Evaluating the role of tokenization in character-level tasks](#). *arXiv preprint arXiv:2508.02591*.
- Catalina Vajiac, Duen Horng Chau, Andreas Oligschlaeger, Rebecca Mackenzie, Pratheeksha Nair, Meng-Chieh Lee, Yifei Li, Namyong Park, Reihaneh Rabbany, and Christos Faloutsos. 2023. [Traficvis: Visualizing organized activity and spatio-temporal patterns for detecting and labeling human](#)

trafficking. *IEEE Transactions on Visualization and Computer Graphics*, 29(1):53–62.

Yibo Wang, Xiangjue Dong, James Caverlee, and Philip S Yu. 2024. Da³: A distribution-aware adversarial attack against language models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Sibo Yi, Yule Liu, Zhen Sun, Tianshuo Cong, Xinlei He, Jiaying Song, Ke Xu, and Qi Li. 2024. Jailbreak attacks and defenses against large language models: A survey. *arXiv preprint arXiv:2407.04295*.

Youliang Yuan, Wenxiang Jiao, Wenxuan Wang, Jentse Huang, Pinjia He, Shuming Shi, and Zhaopeng Tu. 2024. Gpt-4 is too smart to be safe: Stealthy chat with LLMs via cipher. *arXiv preprint arXiv:2308.06463*.

Yunxiang Zhang, Liangming Pan, Samson Tan, and Min-Yen Kan. 2021. Interpreting the robustness of neural nlp models to textual perturbations. *arXiv preprint arXiv:2110.07159*.

Brian Zheng, Alisa Liu, Orevaoghene Ahia, Jonathan Hayase, Yejin Choi, and Noah Smith. 2025a. Broken tokens? your language model can secretly handle non-canonical tokenizations. In *Advances in Neural Information Processing Systems*, volume 38, pages 30322–30349. Curran Associates, Inc.

Muyang Zheng, Yuezhi Yao, Changting Lin, Caihong Kai, Yanxiang Chen, and Zhiqian Liu. 2025b. Mist: Jailbreaking black-box large language models via iterative semantic tuning. *arXiv preprint arXiv:2506.16792*.

Andy Zou, Zifan Wang, J. Zico Kolter, and Matt Fredrikson. 2023. Universal and transferable adversarial attacks on aligned language models. *arXiv preprint arXiv:2307.15043*.

I will give you a unicode character which looks similar to an ASCII letter a-z or A-Z. Your task is to determine what ASCII letter the unicode character most likely resembles. The letter I give is case-sensitive.

Question: What ASCII character is most similar to this unicode character: [EXAMPLE A]

Answer: A

Question: What ASCII character is most similar to this unicode character: [EXAMPLE f]

Answer: f

Question: What ASCII character is most similar to this unicode character: [CHARACTER]

Answer:

Table 2: Prompt used for "What is" letter questions. A similar prompt is used for digits.

You are a phone number decoder. A phone number has been corrupted with unicode characters and punctuation. Your task is to recover just the numerical digits of that phone number. Only respond with the digits of the phone number and no other symbols. Here are a few examples.

Q: What are the digits of this phone number: [EXAMPLE 1]

A: 2144357735

Q: What are the digits of this phone number: [EXAMPLE 2]

A: 7754263233

Q: What are the digits of this phone number: [EXAMPLE 3]

A: 5677381287

Q: What are the digits of this phone number [PHONEWITH-UNICODE]

A:

Table 3: Prompt used for all phone number questions. The examples are mixed with varying digits that have Unicode replacements and/or punctuation noise added, dependent on the specific test.

A Appendix

A.1 Prompts

The prompt templates that we used for the tests in the Fragility Score are presented in Tables 2-6. Several tests have variants that use different numbers of replaced characters or likelihood of noise, but they share the same template prompt. The MMLU baseline test and the MMLU perturbed test, for instance, have the same prompt and just differ in the options text that is presented in the possible answers.

These templates were developed to maximize proper output format and tested across multiple model families to ensure equal opportunity for success.

You are a sentence decoder. Your task is to recover the original sentence from a given string that contains unicode characters that look similar to standard ASCII letters. Only respond with the decoded sentence.

Q: What is this sentence: [EXAMPLE 1]
A: The horse jumped over the fence.
Q: What is this sentence: [EXAMPLE 2]
A: Running is really good for your health!
Q: What is this sentence: [EXAMPLE 3]
A: I enjoy playing video games.
Q: What is this sentence: [QUERY SENT W/ UNICODE]
A:

Table 4: Prompt used for sentence questions.

You are a sentence decoder. Your task is to recover the original sentence from a given string that contains unicode characters that look similar to standard ASCII letters. Follow these three steps: First, break apart the given sentence into individual characters. Second, replace any lookalike unicode characters with their ASCII letters, keeping all others the same. Third, print the sentence in ASCII only.

What is this sentence: [QUERY SENTENCE W/UNICODE]

Table 5: Prompt used for sentence questions with chain-of-thought.

Below is a multiple-choice question. Answer the question with just the letter of the correct answer.

Question: [MMLU QUESTION TEXT]
A. [OPTION A]
B. [OPTION B]
C. [OPTION C]
D. [OPTION D]
Answer:

Table 6: Prompt used for the MMLU tests. The OPTIONS are perturbed or not depending on the test.

A.2 Model Versions and API Usage Cost

The core models evaluated in this work are open-source. We ran the models on two local A100 GPUs, so do not list costs associated with running them. The closed-source GPT models used OpenAI’s API interface with the following cost per model to run all tests on the Fragility Score:

Model	Version	API Cost
GPT-4.1	2025-04-14	\$8.25
GPT-5.4	2026-03-05	\$9.44
GPT-5.5	2026-04-23	\$222.25
4.1-mini	2025-04-14	\$1.73
4o-mini	2024-07-18	\$60.24
5.4-mini	2026-03-17	\$2.90
5-mini	2025-08-07	\$29.60

These API costs change over time, but are listed as the cost to us at the time of running the experiments.

A.3 Model Scores and Consistency

All scores reported in the main Table 1 are from one run using the Fragility Scorer’s static test set. However, to show that the scorer’s tests are not tuned to certain performance, we ran Gemma-1b 12 times with different seeds to generate the data. Gemma-1b across the runs averaged an overall fragility score of 36.3 (matching its reported 36) and a standard deviation of 0.7 (95% CI [35.8,36.8], range [35.5,37.8]). There are over 17k tests in the overall fragility score, so this low variance shows that small changes in tests have little effect on the score.

The number of tests in each of the sub-scores of the Fragility Score are given in Table 7. Each row provides the number of tests for a single test set type, but there are multiple variants of some of them to add up to a total of 17k tests.

A.4 Unicode compared to Blanks

The test for ‘what is this word’ that uses underscores (p_zz_s) is compared to the test with lookalike characters in those same positions. Figure 10 shows the accuracy improvement across all models. Most models show very significant improvement on identifying the word when a lookalike character is given. This shows that models do have the capability of using noisy Unicode, despite its noise hurting other tasks. Performance is worse with Unicode than if it was all plain ASCII, but it’s

Type of Test	# of Tests
MMLU	2400
What is this digit?	334
What is this letter?	1099
What is this word?	300
What is this sentence?	299
Phone Number Single	334
Phone Number Multi	200
Phone Number Noise	500

Table 7: The number of tests in each type of sub-score within the Fragility Score.

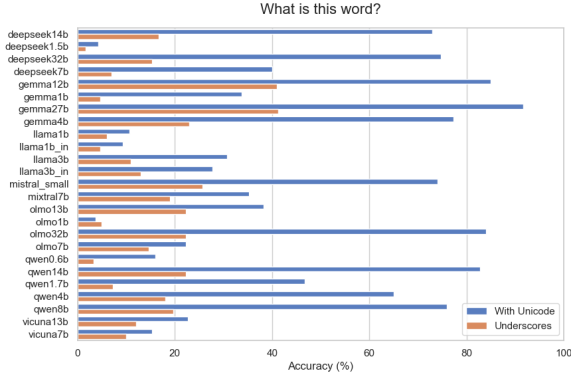


Figure 10: Comparison between the two tests for ‘what is this word?’. The first blue bar shows word prediction when Unicode lookalike characters are in the word. The second orange bar uses underscores instead of lookalikes. Most models show significant gain when lookalikes are included.

clear that the lookalike characters provide some information above just treating them as blanks.

A.5 MMLU Experiment Details

We randomly sampled 2400 questions from the 57 subject MMLU benchmark (Hendrycks et al., 2021) for the LLM tests. For the fragility score itself, we measure MMLU accuracy on the 2400 questions without text changes, and then again but with 20% of the characters in the answers swapped with Unicode lookalike substitutes in the answer options. The score is computed as $(1 - acc_{20}/acc)$. If a model struggles to generate the answer in an extractable format, it is marked incorrect, but the ratio assures the score only penalizes on the noise’s effect.

For further analysis, we varied the percent of characters beyond just the 20% used in the fragility score. We also randomly replaced by 40, and 60%. These results per model are shown in Figure 11. There is a general trend that the models

Model	Score	Unrec	Unrec-20
OLMo 32b	87	0.2%	2.9%
Gemma 27b	81	1.4%	22.2%
Qwen 4b	80	8.0%	11.5%

Table 8: Unrec=Unrecognized Format. Three of the high-scoring models on the MMLU fragility score are listed here to show their percent of output that was an unrecognized format. These models did not always generate a letter A-D for multiple choice. The Unrec columns show the percent of responses that were unrecognized. Unrec-20 is when 20% of the characters were permuted.

experience a big drop when 20% noise is added, and then continual (but not as large) drops with each 40% and 60% after that.

One potential concern is that the MMLU answers are multiple choice letters A-D, and a model may not immediately generate one of the letters. The prompt that we use is given in Table 6. We structured it in a clear Q/A format that is common to text corpora and used in other MMLU work. We did observe models generating explanations instead of letters, or no answer at all. However, the goal is to measure noise effects, so if a model generates unrecognized formats 20% of the time, that’s not of concern to this study. We then measure how noise changes that 20% as well as the correct-to-incorrect flips. A few models and their % blank numbers are given in Table 8. You can see that the small Qwen model actually scores quite high despite an initial high 8% of bad format.

A.6 Punctuation Noise

We include tests for filler noise that does not include Unicode, but instead confounding ASCII punctuation. For phone numbers, this looks like 41.05–5?5-38,53. These tests help compare how model size handles general noise, so that the Unicode lookalike numbers can be better understood in context of a model’s overall capability of representing the data. Figure 12 shows each model with different amounts of noise between characters, ranging from 20 to 100%. The 100% value means each character has some filler noise added immediately after it.

The reduction in performance is visible across all models, but the amount of reduction differs. A few of the models are incomplete because the extra tokenization required expands the input context too far to fit on GPUs.

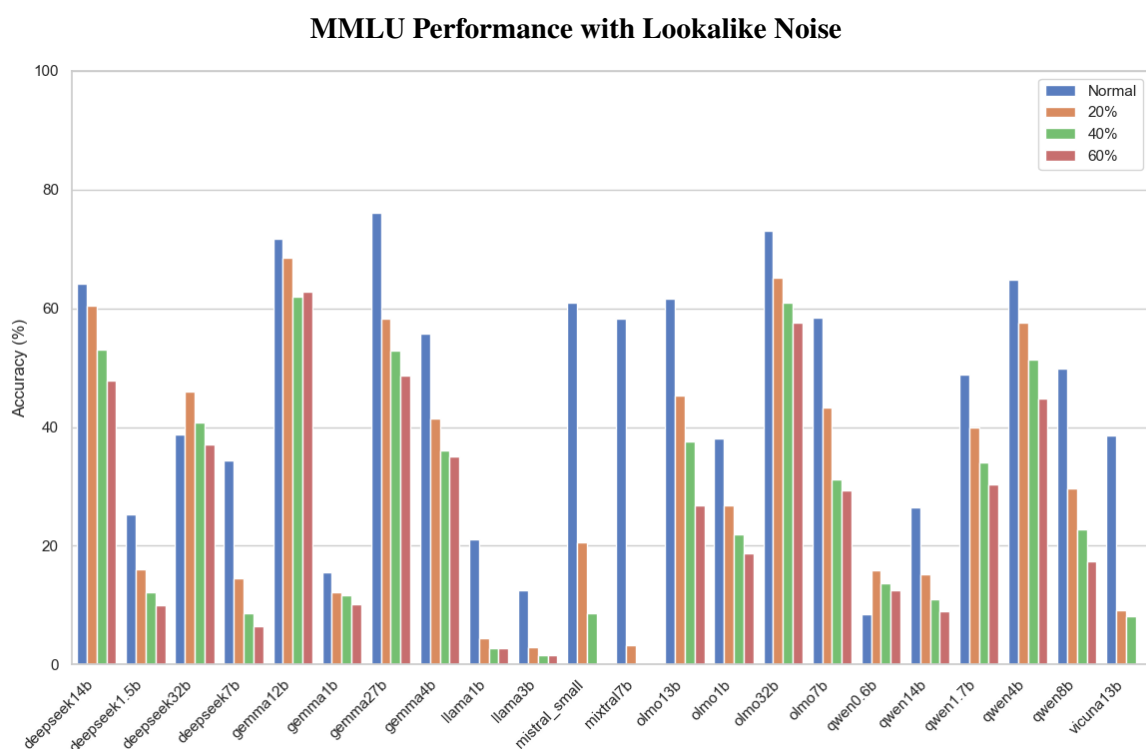


Figure 11: MMLU benchmark results with varying amount of characters replaced by their Unicode lookalikes.

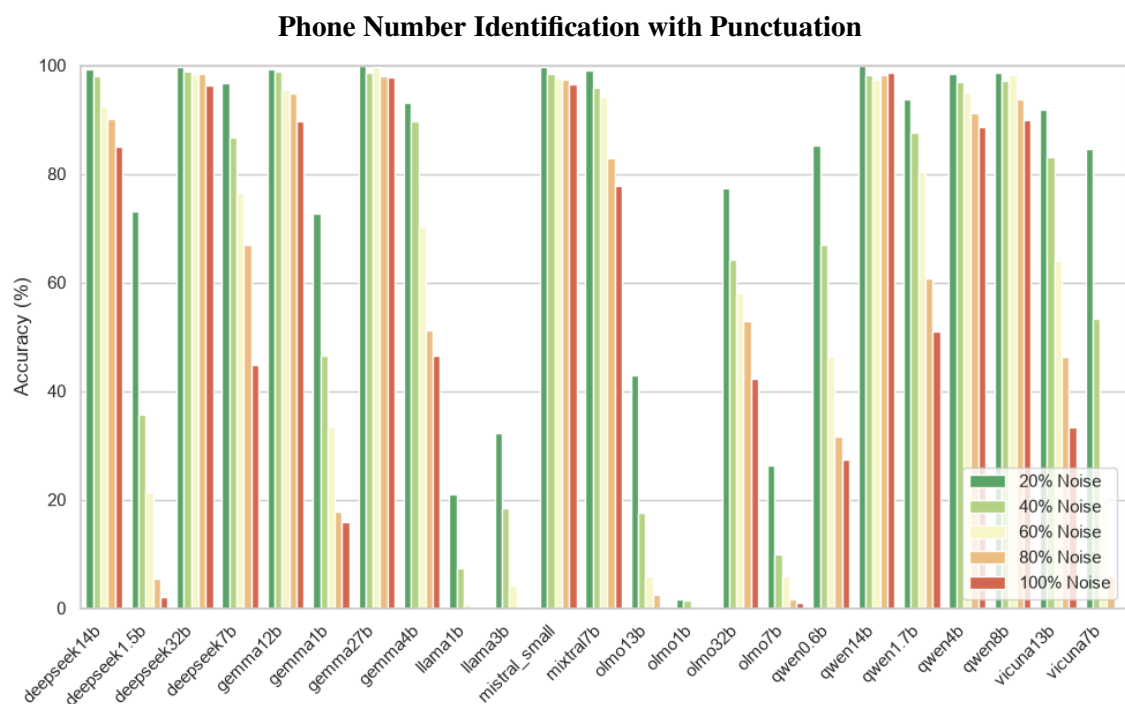


Figure 12: Model results with different amounts of punctuation noise added after each phone number digit.

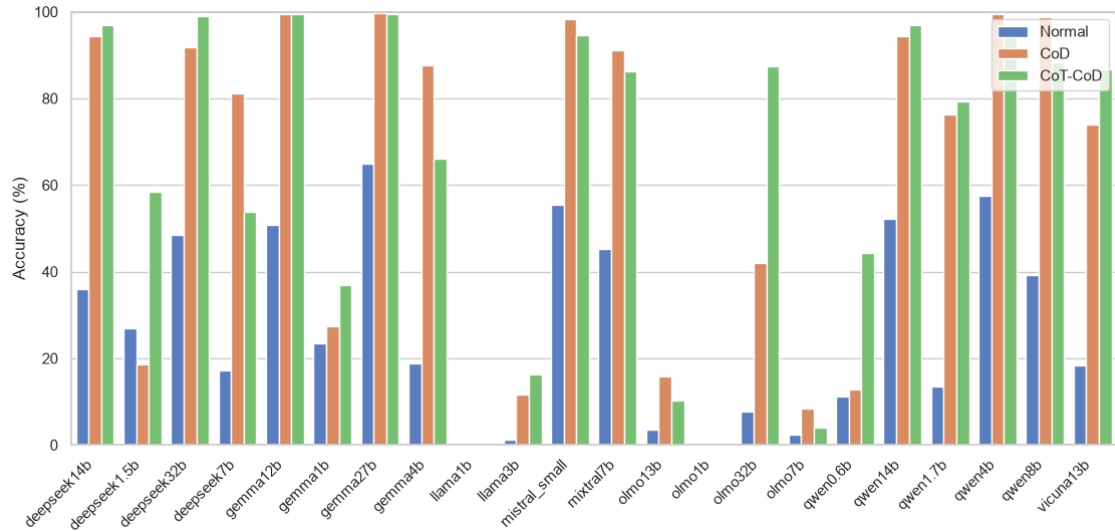


Figure 13: Phone number identification results with chain-of-dictionary (CoD) hints added to the prompts. For each Unicode character in the phone number, its lookalike mapping (α is a) is added to the prompt. The three bars indicate (1) plain prompt, (2) chain-of-dictionary prompt, and (3) chain-of-dictionary with chain-of-thought prompting. Most models see large jumps with CoD and no additional gains (or even hindrance) from CoT.

A.7 Chain-of-Dictionary Effect

The fragility score includes a subcomponent with a small amount of weight that tests the model’s ability to use chain-of-dictionary (CoD) information in the context. If you prompt the model with the gold mapping from lookalike Unicode to its ASCII character, can the model make correct use of the information? We include this in the phone number identification test and Figure 13 shows the per-model improvement with CoD clues added to the prompt. Most models see large jumps with CoD, with the exception of the smallest (1b, 3b, etc.). This suggests that most models have the capacity to understand lookalikes, but they don’t have the token representation nor proper training.